



DATAPROJECT

Lógica. Consultas de SQL

Lidia Utrilla Olivera

10 de junio 2026



El presente documento recoge el desarrollo de un proyecto de consultas SQL realizado sobre una base de datos relacional que simula el funcionamiento de una tienda de alquiler de películas. El objetivo principal del trabajo ha sido aplicar los conocimientos adquiridos sobre bases de datos y lenguaje SQL mediante la elaboración de 64 consultas destinadas a extraer, analizar y gestionar la información almacenada.

La base de datos está compuesta por diversas tablas relacionadas entre sí, entre las que destacan las que almacenan información sobre películas, actores, categorías, clientes, empleados, alquileres, pagos, inventario y establecimientos. Gracias a esta estructura relacional, es posible realizar consultas que permiten obtener información relevante sobre la actividad de la tienda, los hábitos de los clientes y las características del catálogo disponible.

A lo largo del proyecto se han utilizado diferentes sentencias y cláusulas SQL, incluyendo operaciones de selección, filtrado, ordenación, agrupación, funciones de agregación y combinaciones de tablas mediante relaciones. Asimismo, algunas consultas requieren un mayor nivel de complejidad, incorporando subconsultas y cálculos sobre los datos obtenidos.

El desarrollo de estas 64 consultas ha permitido reforzar la comprensión del modelo relacional y demostrar la capacidad de obtener información útil a partir de grandes volúmenes de datos estructurados. En las páginas siguientes se presentan las consultas realizadas, acompañadas de sus resultados

Consultas SQL

1. Crea el esquema de la BBDD.

El esquema entidad relación de la base de datos se adjunta a continuación. Además, se adjunta el código en SQL de algunas tablas. En una base de datos relacional, el esquema define:

1. Las tablas.
2. Sus columnas y tipos de datos.
3. Las claves primarias (PK).
4. Las claves foráneas (FK).
5. Las restricciones (NOT NULL, UNIQUE, CHECK, etc.).

```
-- TABLE FILM
CREATE TABLE film (
  film_id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  release_year INTEGER,
  language_id INTEGER NOT NULL,
  original_language_id INTEGER,
  rental_duration INTEGER NOT NULL,
  rental_rate NUMERIC(4,2) NOT NULL,
```

```

length INTEGER,
replacement_cost NUMERIC(5,2),
rating VARCHAR(10),

CONSTRAINT fk_film_language
FOREIGN KEY (language_id)
REFERENCES language(language_id),

CONSTRAINT fk_film_original_language
FOREIGN KEY (original_language_id)
REFERENCES language(language_id)
);

-- TABLE ACTOR
CREATE TABLE actor (
    actor_id SERIAL PRIMARY KEY,
    first_name VARCHAR(45) NOT NULL,
    last_name VARCHAR(45) NOT NULL
);

-- TABLE FILM_aCTOR
CREATE TABLE film_actor (
    actor_id INTEGER NOT NULL,
    film_id INTEGER NOT NULL,

    PRIMARY KEY (actor_id, film_id),

    FOREIGN KEY (actor_id)
    REFERENCES actor(actor_id),

    FOREIGN KEY (film_id)
    REFERENCES film(film_id)
);

-- TABLE CATEGORY
CREATE TABLE category (
    category_id SERIAL PRIMARY KEY,
    name VARCHAR(25) NOT NULL
);

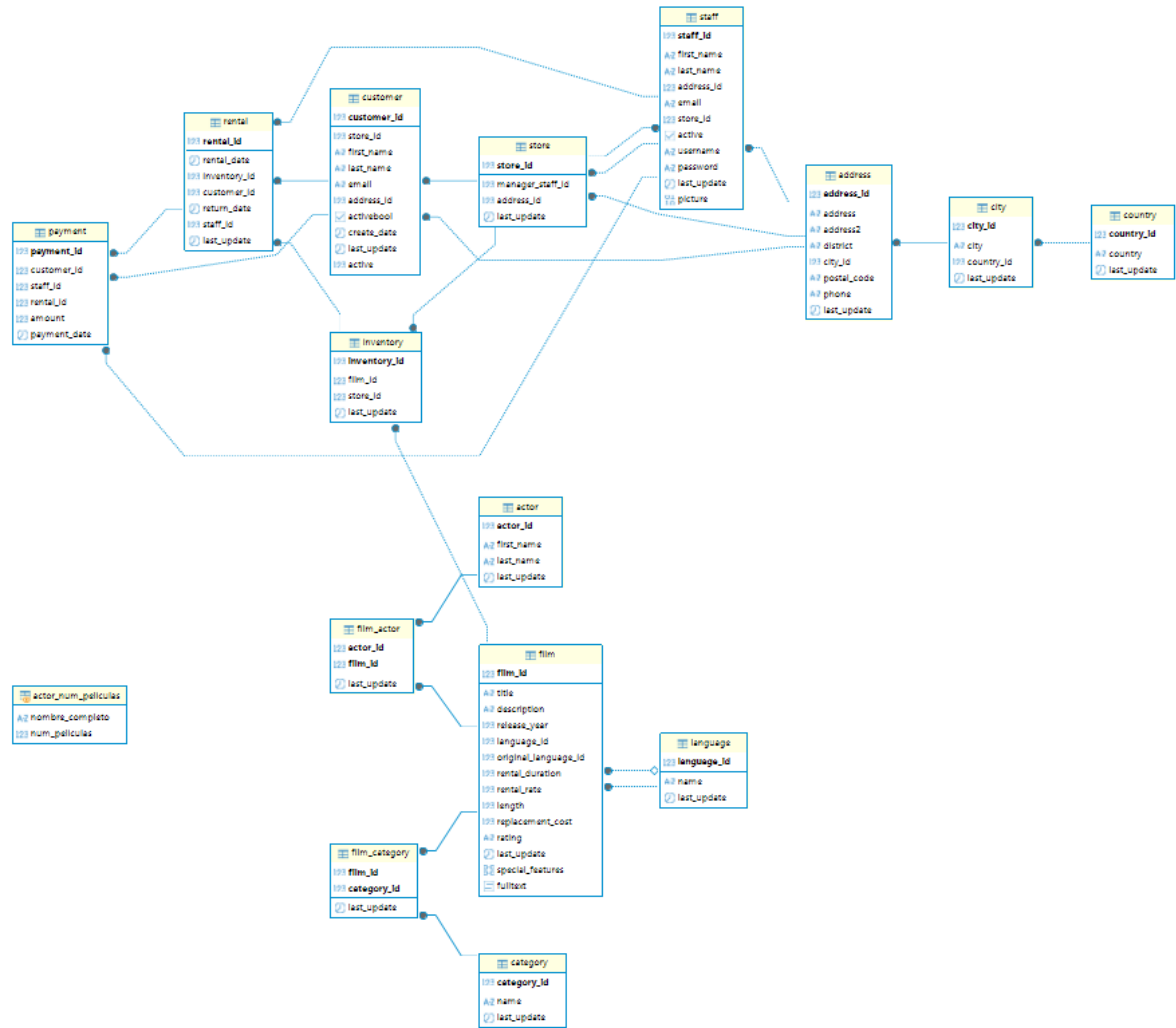
-- TABLE FILM CATEGORY
CREATE TABLE film_category (
    film_id INTEGER NOT NULL,
    category_id INTEGER NOT NULL,

    PRIMARY KEY (film_id, category_id),

    FOREIGN KEY (film_id)
    REFERENCES film(film_id),

    FOREIGN KEY (category_id)
    REFERENCES category(category_id)
);

```



2. Muestra los nombres de todas las películas con una clasificación por edades de 'R'.

La clasificación "R" es un sistema de calificación por edades utilizado principalmente por la asociación cinematográfica de Estados Unidos.

R = Restricted (Restringida) → Significa que los menores de 17 años no deberían ver la película solos. Deben ir acompañados por un padre, madre o tutor.

La tabla film presenta la columna rating, que almacena la clasificación por edades de cada película.

```
select "title"
from film f
where rating = 'R';
```

3. Encuentra los nombres de los actores que tengan un "actor_id" entre 30 y 40.

```
Select "first_name"
from actor a
where "actor_id">=30 and "actor_id"<=40;
```

```
select "first_name"
from actor a
where "actor_id" between 30 and 40;
```

4. Obtén las películas cuyo idioma coincide con el idioma original.

La query que nos solicitan sería la siguiente:

```
select "title"
from film f
where "language_id" = "original_language_id";
```

Sin embargo, el resultado de la consulta es 0 filas dado que "original_language_id" es siempre null. Se revisa la tabla idiomas, la cual muestra que hay 6 idiomas diferentes.

5. Ordena las películas por duración de forma ascendente.

```
select "title", "length"
from film f
order by "length" asc;
```

6. Encuentra el nombre y apellido de los actores que tengan 'Allen' en su apellido.

```
select
    "first_name",
    "last_name"
from actor a
where "last_name" like '%ALLEN%';
```

7. Encuentra la cantidad total de películas en cada clasificación de la tabla "film" y muestra la clasificación junto con el recuento.

```
select
    "rating" as "clasificacion",
    count(*) as "total_películas"
from "film"
group by ("rating");
```

En esta query el count(*), cuenta todas las filas de la tabla incluidos los valores NULL. En caso de no querer que cuente los valores NULL empleamos COUNT(columna).

```
select
    "rating" as "clasificacion",
    count("rating") as "total_películas"
from "film"
group by ("rating");
```

8. Encuentra el título de todas las películas que son 'PG-13' o tienen una duración mayor a 3 horas en la tabla film.

```
select
    "title",
from film f
where "rating"='PG-13' or "length">180;
```

9. Encuentra la variabilidad de lo que costaría reemplazar las películas.

```
select
    stddev("replacement_cost") as "desviacion_estandar"
from film f;
```

10. Encuentra la mayor y menor duración de una película de nuestra BBDD.

```
SELECT
    Min("length") as "mínima_duración",
```

```

    max("length") as "máxima_duración"
from "film"

```

Esta query nos indica únicamente los valores máximos y mínimos. Para conocer que película es la más corta y la más larga:

```

--Título de la película más larga:
SELECT
    "title",
    "length"
FROM film
WHERE "length" = (
    SELECT MAX("length")
    FROM "film"
);
--Título de la película más corta:
SELECT
    "title",
    "length"
FROM film
WHERE "length" = (
    SELECT MIN("length")
    FROM "film"
);

```

11. Encuentra lo que costó el antepenúltimo alquiler ordenado por día.

```

select
    r.rental_id,
    r.rental_date,
    p.amount
FROM "rental" r
join "payment" p
    on r.rental_id = p.rental_id
order by "rental_date" desc, r.rental_id desc
limit 1 offset 2;

```

Crea un poco de confusión que haya diferentes pagos para una misma fecha, por ello he ordenado también por rental_id. Obtengo el tercer alquiler más reciente.

12. Encuentra el título de las películas en la tabla “film” que no sean ni ‘NC-17’ ni ‘G’ en cuanto a su clasificación.

```

SELECT
    "title"
FROM film
where "rating" not in ('NC-17','G');

```

13. Encuentra el promedio de duración de las películas para cada clasificación de la tabla film y muestra la clasificación junto con el promedio de duración.

```

select
    "rating" as clasificacion,
    AVG("length") as duracion_promedio
FROM film
group by "rating";

```

14. Encuentra el título de todas las películas que tengan una duración mayor a 180 minutos.

```
select
    "title"
from film
where "length" > 180;
```

15. ¿Cuánto dinero ha generado en total la empresa?

```
select
    sum("amount") as ingresos_totales
from "payment";
```

16. Muestra los 10 clientes con mayor valor de id.

```
select
    "first_name" as nombre,
    "last_name" as apellido
from "customer"
order by "customer_id" desc
limit 10;
```

17. Encuentra el nombre y apellido de los actores que aparecen en la película con título 'Egg Igbby'.

```
select
    a.first_name,
    a.last_name
from film_actor fa
join actor a
    on fa.actor_id = a.actor_id
join film f
    on fa.film_id = f.film_id
where f.title='EGG IGBY';
```

18. Selecciona todos los nombres de las películas únicos.

```
SELECT DISTINCT "title"
FROM "film";
```

19. Encuentra el título de las películas que son comedias y tienen una duración mayor a 180 minutos en la tabla "film".

```
select f.title
from film f
join film_category fc
    on f.film_id = fc.film_id
join category c
    on fc.category_id = c.category_id
where c.name = 'Comedy' and f.length >180;
```

20. Encuentra las categorías de películas que tienen un promedio de duración superior a 110 minutos y muestra el nombre de la categoría junto con el promedio de duración.

```
select
    c.name as categoría,
    round(avg(f.length),2) as duracion_promedio
from film f
```

```

join film_category fc
  on f.film_id = fc.film_id
join category c
  on fc.category_id = c.category_id
group by c.category_id, c.name
having AVG(f.length )>110

```

Empleo HAVING porque estamos filtrando un resultado agregado.

21. ¿Cuál es la media de duración del alquiler de las películas?

```

select
  avg(rental_duration) as media_duracion_alquiler
from film;

```

22. Crea una columna con el nombre y apellidos de todos los actores y actrices.

```

select
  concat("first_name",' ','last_name") as nombre_completo
from actor;

```

23. Números de alquiler por día, ordenados por cantidad de alquiler de forma descendente.

```

select
  DATE(rental_date) as dia,
  count (*) as numero_alquileres
from rental
GROUP BY DATE(rental_date)
ORDER BY numero_alquileres desc;

```

Empleo DATE para eliminar la hora de la fecha de alquiler. Count (*) cuenta todas las filas.

24. Encuentra las películas con una duración superior al promedio.

```

select
  "title"
from film
where length > (
  select
    avg(length) as promedio_duracion
  from film
);

```

Incorporo una subconsulta.

25. Averigua el número de alquileres registrados por mes.

```

select
  TO_CHAR(rental_date, 'YYYY-MM') as mes,
  count (*) as numero_alquileres
from rental
GROUP BY TO_CHAR(rental_date, 'YYYY-MM')
ORDER BY mes;

```

Empleo TO_CHAR para agrupar por mes la fecha de alquiler. Count (*) cuenta todas las filas.

26. Encuentra el promedio, la desviación estándar y varianza del total pagado.

```
select
  round(avg(amount),3) as promedio_pagado,
  round(stddev(amount),3) as desviacion_estandar,
  round(variance(amount),3) as varianza
from payment;
```

27. ¿Qué películas se alquilan por encima del precio medio?

```
select title
from film
where rental_rate > (
  select
    avg(rental_rate) as precio
  from film
);
```

Incorpora una subconsulta.

28. Muestra el id de los actores que hayan participado en más de 40 películas.

```
SELECT
  a.actor_id,
  a.first_name,
  a.last_name
FROM actor a
JOIN film_actor fa
  ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name
HAVING COUNT(fa.film_id) > 40;
```

29. Obtener todas las películas y, si están disponibles en el inventario, mostrar la cantidad disponible.

```
select
  f.title,
  COUNT(i.inventory_id) as cantidad_disponible
from film f
left join inventory i
  on f.film_id = i.film_id
GROUP BY f.film_id, f.title;
```

30. Obtener los actores y el número de películas en las que ha actuado.

```
select
  concat(a.first_name, ' ', a.last_name) as nombre_completo,
  COUNT(fa.film_id) as numero_peliculas_actuacion
from actor a
left join film_actor fa
  on a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name;
```

31. Obtener todas las películas y mostrar los actores que han actuado en ellas, incluso si algunas películas no tienen actores asociados.

```
select
  f.title as pelicula,
  concat(a.first_name, ' ', a.last_name) as nombre_completo
from film f
left join film_actor fa
  on f.film_id = fa.film_id
```

```

left join actor a
on fa.actor_id = a.actor_id

```

32. Obtener todos los actores y mostrar las películas en las que han actuado, incluso si algunos actores no han actuado en ninguna película.

```

select
    concat(a.first_name, ' ', a.last_name) as nombre_completo,
    f.title as pelicula
FROM actor a
LEFT JOIN film_actor fa
    ON a.actor_id = fa.actor_id
LEFT JOIN film f
    ON fa.film_id = f.film_id;

```

33. Obtener todas las películas que tenemos y todos los registros de alquiler.

```

SELECT
    f.title AS pelicula,
    r.rental_id,
    r.rental_date
FROM film f
LEFT JOIN inventory i
    ON f.film_id = i.film_id
LEFT JOIN rental r
    ON i.inventory_id = r.inventory_id

UNION

SELECT
    f.title AS pelicula,
    r.rental_id,
    r.rental_date
FROM rental r
LEFT JOIN inventory i
    ON r.inventory_id = i.inventory_id
LEFT JOIN film f
    ON i.film_id = f.film_id;

```

La primera parte del código nos muestra todas las películas con o sin alquiler, y la segunda parte todos los alquileres con o sin película.

34. Encuentra los 5 clientes que más dinero se hayan gastado con nosotros.

```

select
    concat(c.first_name, ' ', c.last_name) as nombre_completo,
    SUM(p.amount) as total_gastado
from customer c
left join payment p
    on c.customer_id = p.customer_id
group by c.first_name, c.last_name, p.amount
order by total_gastado desc
limit 5;

```

35. Selecciona todos los actores cuyo primer nombre es 'Johnny'.

```

select
    concat(first_name, ' ', last_name) as nombre_completo
from actor
where first_name = 'JOHNNY';

```

36. Renombra la columna “first_name” como Nombre y “last_name” como Apellido.

```
select
    first_name as Nombre,
    last_name as Apellido
from actor a ;
```

37. Encuentra el ID del actor más bajo y más alto en la tabla actor.

```
select
    min(actor_id) as id_mas_bajo,
    max(actor_id) as id_mas_alto
from actor;
```

38. Cuenta cuántos actores hay en la tabla “actor”.

```
select
    count(actor_id) as total_actores
from actor;
```

39. Selecciona todos los actores y ordénalos por apellido en orden ascendente.

```
select
    first_name,
    last_name
from actor
order by (last_name)asc;
```

40. Selecciona las primeras 5 películas de la tabla “film”.

```
select title
from film
limit 5;
```

41. Agrupa los actores por su nombre y cuenta cuántos actores tienen el mismo nombre. ¿Cuál es el nombre más repetido?

```
select
    count(actor_id ),
    first_name
from actor
group by first_name;
```

42. Encuentra todos los alquileres y los nombres de los clientes que los realizaron.

```
SELECT
    r.rental_id,
    r.rental_date,
    concat(first_name, ' ', last_name) as nombre_completo
FROM rental r
JOIN customer c
    ON r.customer_id = c.customer_id;
```

43. Muestra todos los clientes y sus alquileres si existen, incluyendo aquellos que no tienen alquileres.

```
select
    c.customer_id,
    concat(first_name, ' ', last_name) as nombre_completo,
    r.rental_id,
```

```

        r.rental_date
FROM customer c
left JOIN rental r
    ON c.customer_id = r.customer_id ;

```

44. Realiza un CROSS JOIN entre las tablas film y category. ¿Aporta valor esta consulta? ¿Por qué? Deja después de la consulta la contestación.

```

SELECT
    f.title,
    c.name AS categoria
FROM film f
CROSS JOIN category c;

```

El CROSS JOIN no aporta valor para este caso porque genera todas las combinaciones posibles entre películas y categorías, sin tener en cuenta las relaciones reales existentes en la base de datos. Produce un producto cartesiano que aumenta mucho el número de filas y puede generar información irrelevante o engañosa.

45. Encuentra los actores que han participado en películas de la categoría 'Action'.

SELECT distinct --evita que un actor aparezca varias veces si ha participado en varias películas de acción.

```

    a.first_name,
    a.last_name
FROM actor a
JOIN film_actor fa
    ON a.actor_id = fa.actor_id
JOIN film_category fc
    ON fa.film_id = fc.film_id
JOIN category c
    ON fc.category_id = c.category_id
WHERE c.name = 'Action';

```

46. Encuentra todos los actores que no han participado en películas.

```

select
    a.first_name,
    a.last_name
FROM actor a
LEFT JOIN film_actor fa
    ON a.actor_id = fa.actor_id
WHERE fa.actor_id IS NULL;

```

47. Selecciona el nombre de los actores y la cantidad de películas en las que han participado.

```

select
    concat(first_name, ' ', last_name) as nombre_completo,
    COUNT( fa.film_id) as cantidad_peliculas
FROM actor a
LEFT JOIN film_actor fa
    ON a.actor_id = fa.actor_id
group by a.first_name, a.last_name;

```

48. Crea una vista llamada “actor_num_peliculas” que muestre los nombres de los actores y el número de películas en las que han participado.

```
CREATE VIEW actor_num_peliculas AS
SELECT
    concat(first_name, ' ', last_name) as nombre_completo,
    COUNT(fa.film_id) AS num_peliculas
FROM actor a
LEFT JOIN film_actor fa
    ON a.actor_id = fa.actor_id
GROUP BY a.first_name, a.last_name;
--Consultar la vista:
SELECT *
FROM actor_num_peliculas;
```

49. Calcula el número total de alquileres realizados por cada cliente.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(r.rental_id) AS total_alquileres
FROM customer c
LEFT JOIN rental r
    ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

50. Calcula la duración total de las películas en la categoría 'Action'.

```
select
    sum(f.length ) as duracion_total
from film f
join film_category fc
    on f.film_id = fc.film_id
join category c
    on fc.category_id = c.category_id
where c."name" = 'Action'
```

51. Crea una tabla temporal llamada “cliente_rentas_temporal” para almacenar el total de alquileres por cliente.

```
CREATE TEMPORARY TABLE cliente_rentas_temporal AS
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(r.rental_id) AS total_alquileres
FROM customer c
LEFT JOIN rental r
    ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
--Consultar tabla temporal:
SELECT *
FROM cliente_rentas_temporal;
```

52. Crea una tabla temporal llamada “peliculas_alquiladas” que almacene las películas que han sido alquiladas al menos 10 veces.

```
CREATE TEMPORARY TABLE peliculas_alquiladas AS
select
    f.title,
```

```

        count(r.rental_id ) as total_alquileres
from film f
join inventory i
    on f.film_id = i.film_id
join rental r
    on i.inventory_id = r.inventory_id
GROUP BY f.title
HAVING COUNT(r.rental_id) >= 10;
--Consultar tabla:
select*
from peliculas_alquiladas

```

53. Encuentra el título de las películas que han sido alquiladas por el cliente con el nombre 'Tammy Sanders' y que aún no se han devuelto. Ordena los resultados alfabéticamente por título de película.

```

SELECT f.title
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
WHERE c.first_name = 'TAMMY'
      AND c.last_name = 'SANDERS'
      AND r.return_date IS NULL
ORDER BY f.title;

```

55. Encuentra el nombre y apellido de los actores que han actuado en películas que se alquilaron después de que la película 'Spartacus Cheaper' se alquilara por primera vez. Ordena los resultados alfabéticamente por apellido.

```

SELECT DISTINCT
    a.first_name,
    a.last_name
FROM actor a
JOIN film_actor fa
    ON a.actor_id = fa.actor_id
JOIN film f
    ON fa.film_id = f.film_id
JOIN inventory i
    ON f.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
WHERE r.rental_date > (
    SELECT MIN(r2.rental_date)
    FROM film f2
    JOIN inventory i2
        ON f2.film_id = i2.film_id
    JOIN rental r2
        ON i2.inventory_id = r2.inventory_id
    WHERE f2.title = 'SPARTACUS CHEAPER'
)
ORDER BY a.last_name, a.first_name;

```

56. Encuentra el nombre y apellido de los actores que no han actuado en ninguna película de la categoría 'Music'.

```
SELECT
    a.first_name,
    a.last_name
FROM actor a
WHERE NOT EXISTS (
    SELECT *
    FROM film_actor fa
    JOIN film_category fc
        ON fa.film_id = fc.film_id
    JOIN category c
        ON fc.category_id = c.category_id
    WHERE fa.actor_id = a.actor_id
        AND c.name = 'Music'
)
ORDER BY a.last_name, a.first_name;
```

57. Encuentra el título de todas las películas que fueron alquiladas por más de 8 días.

```
SELECT DISTINCT
    f.title
FROM film f
JOIN inventory i
    ON f.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
WHERE (r.return_date::date - r.rental_date::date) > 8;
```

58. Encuentra el título de todas las películas que son de la misma categoría que 'Animation'.

```
SELECT
    f.title
FROM film f
JOIN film_category fc
    ON f.film_id = fc.film_id
JOIN category c
    ON fc.category_id = c.category_id
WHERE c.name = 'Animation';
```

59. Encuentra los nombres de las películas que tienen la misma duración que la película con el título 'Dancing Fever'. Ordena los resultados alfabéticamente por título de película.

```
SELECT
    title
FROM film
WHERE length = (
    SELECT length
    FROM film
    WHERE title = 'DANCING FEVER'
)
ORDER BY title;
```

60. Encuentra los nombres de los clientes que han alquilado al menos 7 películas distintas. Ordena los resultados alfabéticamente por apellido.

```
SELECT
    c.first_name,
    c.last_name
FROM customer c
JOIN rental r
    ON c.customer_id = r.customer_id
JOIN inventory i
    ON r.inventory_id = i.inventory_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(DISTINCT i.film_id) >= 7
ORDER BY c.last_name, c.first_name;
```

61. Encuentra la cantidad total de películas alquiladas por categoría y muestra el nombre de la categoría junto con el recuento de alquileres.

```
SELECT
    c.name AS categoria,
    COUNT(r.rental_id) AS total_alquileres
FROM category c
JOIN film_category fc
    ON c.category_id = fc.category_id
JOIN inventory i
    ON fc.film_id = i.film_id
JOIN rental r
    ON i.inventory_id = r.inventory_id
GROUP BY c.category_id, c.name
ORDER BY total_alquileres DESC;
```

62. Encuentra el número de películas por categoría estrenadas en 2006.

```
SELECT
    c.name AS categoria,
    COUNT(f.film_id) AS num_peliculas
FROM category c
JOIN film_category fc
    ON c.category_id = fc.category_id
JOIN film f
    ON fc.film_id = f.film_id
WHERE f.release_year = 2006
GROUP BY c.category_id, c.name;
```

63. Obtén todas las combinaciones posibles de trabajadores con las tiendas que tenemos.

```
SELECT
    s.staff_id,
    s.first_name,
    s.last_name,
    st.store_id
FROM staff s
CROSS JOIN store st;
```


64. Encuentra la cantidad total de películas alquiladas por cada cliente y muestra el ID del cliente, su nombre y apellido junto con la cantidad de películas alquiladas.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(r.rental_id) AS total_alquileres
FROM customer c
LEFT JOIN rental r
    ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_alquileres DESC;
```